Programación Concurrente y Paralela

Ejercicio Mensajes

2do Cuatrimestre 2026

Ejercicio 1. Queremos modelar a una persona que hace una transmisión de streaming, su plataforma, y a la gente que está subscripta a ella. Modelaremos el stream como una secuencia de elementos `Package` (que, en Go, serían un `struct{ contenido : string }`, para poder imprimirlo). La persona streamer tiene su `plataformaStream` que se encarga de enviar cada `Package` generado por la persona streamer a la gente que esté suscripta a la transmisión, hasta que esta se termine.

Implementar utilizando comunicación por medio de mensajes:

a) Un thread `streamer` y un thread `plataformaStream`. `plataformaStream` recibe una lista de canales de `Package` suscriptos a la transmisión. Ambos threads tienen un canal en común por el que `streamer` envía sincrónicamente cada package cada 0.5 segundos y `plataformaStream` lo replica a todos los canales en la lista de suscripciones. Por lo pronto, `streamer` genera `Package`s indefinidamente.

b) Modifique la solución anterior para que ahora `plataformaStream` no reciba la lista de suscripciones por parámetro, sino que ésta comience vacía y las personas suscriptoras deban unirse a ella por medio de mensajes. Suponga que habrán exactamente 3 suscripciones, y haga que el thread `streamer` comience la transmisión una vez las mismas hayan llegado. Agregue el thread `suscriptor` que se agregue como suscriptor e imprima los `Packages` (o bien sus contenido) a medida que vayan llegando.

c) Modifique la solución para que ahora puedan seguir llegando suscripciones una vez comenzada la transmisión, y que además sea posible dar de baja suscripciones (puede suponer que las primeras 3 acciones que llegarán son 3 suscripciones). Para el thread `suscriptor`, haga que reciba alguna cantidad aleatoria de paquetes (se puede suponer mayor que 1) y se desuscriba del stream. Puede suponer que el espacio entre `Package` y `Package` es suficiente como para que en el medio se pueda hacer el envío a los canales suscriptores y aplicar las suscripciones/desuscripciones de haberlas.

d) Finalmente, modifique la solución para que si, durante el lapso de la transmisión sucede que la cantidad de suscripciones es cero, se informe a la persona streamer y esta detenga la transmisión (no debería mandar más que un Package desde la última desuscripción hasta la anulación del stream, suponga que una vez que las subscripciones llegaron a 0, no habrán más suscripciones nuevas). Modele el fin de la transmisión como el cierre del canal entre `streamer` y `plataformaStream`.